
SISTEMA DE GESTIÓN DE BIBLIOTECA MEDIANTE ESTRUCTURAS DE DATOS NO LINEALES EN C# ASP.NET CORE MVC

Carnet 202300379 – Mónica Sofia Boche Figueroa

Resumen

El presente informe describe el desarrollo e implementación de una plataforma web creada utilizando C# y .NET Core, siguiendo el patrón arquitectónico Model-View-Controller (MVC). El sistema tiene como objetivo facilitar la administración y organización de un catálogo de biblioteca.

Para organizar la información, se implementaron manualmente diferentes Tipos de Datos Abstractos (TDA) no lineales. Se utilizó un Árbol N-Ario para organizar las categorías de manera jerárquica y dinámica, mientras que un Árbol AVL permite almacenar y ordenar los libros según su ISBN, logrando búsquedas eficientes con una complejidad de $O(\log n)$.

Además, el sistema permite procesar archivos XML de manera incremental, mantener la información almacenada en memoria mediante un patrón Singleton y generar representaciones gráficas interactivas de las estructuras de datos. Para esta visualización se utilizan el lenguaje DOT de Graphviz y la biblioteca Viz.js, integrados dentro del entorno web.

Palabras clave

1. Árbol AVL
2. Árbol N-Ario
3. Tipo de Dato Abstracto

4. ASP.NET Core MVC
5. Graphviz

Abstract

This report presents the design and implementation of a web platform built on C# and .NET Core using the Model-View-Controller (MVC) architectural pattern for efficient library catalog management. The solution addresses the hierarchical organization of categories and the balanced management of books through custom non-linear Abstract Data Types (ADT) implemented without framework collection generics. An N-ary Tree is utilized to structure dynamic categories, while an AVL Tree stores books indexed by ISBN, guaranteeing optimal $O(\log n)$ lookup times. The system features incremental XML file processing, singleton memory persistence, and interactive graphical rendering using Graphviz DOT syntax and Viz.js on the web interface.

Keywords

1. AVL Tree
2. N-ary Tree
3. Abstract Data Type
4. ASP.NET Core MVC
5. Graphviz

Introducción

En el área de la computación, trabajar con grandes cantidades de información organizada de forma jerárquica y lograr búsquedas rápidas es un reto importante. Las estructuras de datos lineales, como los arreglos y las listas enlazadas, pueden volverse poco eficientes cuando la cantidad de información aumenta considerablemente. Por esta razón, los Tipos de Datos Abstractos (TDA) no lineales permiten manejar los datos de una manera más eficiente, gracias a características como la ramificación y el equilibrio de sus estructuras.

En este proyecto se desarrolló un sistema web que utiliza Árboles N-Arios para organizar las categorías de la biblioteca de forma jerárquica y sin un límite fijo de niveles. También se implementó un Árbol AVL para almacenar los libros y mantenerlos ordenados y equilibrados, facilitando así las búsquedas.

Todo el sistema fue desarrollado en C#, evitando el uso de las colecciones nativas de la plataforma, como System.Collections.Generic. Esto permitió implementar manualmente la administración de los datos mediante nodos y referencias personalizadas, con el objetivo de comprender y aplicar directamente el funcionamiento de estas estructuras de datos.

1. Tipos de Datos Abstractos No Lineales

Un Tipo de Dato Abstracto (TDA) define un conjunto de datos y las operaciones permitidas sobre ellos, ocultando la complejidad interna de su implementación al usuario final.

1.1. Árbol N-Ario (Jerarquía de Categorías)

Un Árbol N-ario es una estructura de datos no lineal en la que cada nodo puede tener varios hijos, sin establecer una cantidad específica. En este proyecto, esta estructura se utiliza para organizar las categorías y subcategorías de la biblioteca de forma jerárquica.

1.2. Árbol AVL (Catálogo Balanceado de Libros)

Un Árbol AVL es un árbol binario de búsqueda auto-balanceado. La propiedad fundamental radica en que las alturas de los dos subárboles de cualquier nodo difieren como máximo en 1.

$$Factro\ equilibrio = Altura\ (Der) - Altura\ (Izq)$$

Si el factor de equilibrio se desvía a -2 o +2, se ejecutan rotaciones (Simples a la Izquierda/Derecha o Dobles Izquierda-Derecha/Derecha-Izquierda) para reestructurar el árbol en tiempo $O(\log n)$.

2. Arquitectura Web y Persistencia (ASP.NET Core MVC)

La aplicación adopta el patrón Modelo-Vista-Controlador (MVC):

- **Modelos:** Contienen las entidades de datos (Libro, NodoCategoria, NodoLibro).
- **Controladores:** Unidades lógicas que coordinan las operaciones (CargaController, CatalogoController, HomeController).
- **Vistas:** Plantillas Razor (.cshtml) encargadas de renderizar la interfaz de usuario con HTML5 y Bootstrap.

2.1. Persistencia Singleton

Para poder cargar los archivos XML de manera progresiva y mantener la información del catálogo entre las diferentes solicitudes HTTP, las raíces de los árboles se guardan en variables estáticas dentro de la memoria de la aplicación web. Esto permite conservar los datos y el estado de las estructuras mientras el sistema continúa funcionando.

3. Generación y Visualización de Gráficos (Graphviz & Viz.js)

Para mostrar gráficamente las estructuras de datos del sistema, se implementaron métodos recursivos que convierten la información interna de los árboles al formato DOT utilizado por Graphviz.

En lugar de instalar programas externos en el servidor para generar los gráficos, las cadenas en formato DOT se envían directamente a la vista web mediante ViewBag. Luego, estas cadenas son procesadas en el navegador utilizando la biblioteca de JavaScript Viz.js, que se encarga de generar los gráficos en formato SVG. De esta manera, las representaciones pueden visualizarse de forma interactiva y mantener una buena calidad al cambiar su tamaño.

4. Procesamiento de Archivos XML

La carga masiva se realiza mediante el parsing de documentos XML estructurados empleando System.Xml. El algoritmo lee recursivamente las etiquetas de categorías y libros, ejecutando inserciones automáticas tanto en el Árbol N-Ario como en el Árbol AVL global, manejando casos de duplicados mediante la actualización de atributos.

Conclusiones

1. La creación manual de los Árboles AVL y N-Arios en C# permitió trabajar sin utilizar las colecciones genéricas propias de .NET. Esto ayudó a comprender mejor el manejo de la memoria y permitió obtener estructuras eficientes para realizar búsquedas en un tiempo de $O(\log n)$.
2. El uso de la arquitectura ASP.NET Core MVC, junto con el almacenamiento de la información en memoria mediante variables estáticas, permitió conservar los datos del sistema mientras se cargaban de manera progresiva varios archivos XML.

3. La utilización del lenguaje DOT junto con la biblioteca Viz.js permitió representar visualmente las estructuras de datos directamente en el navegador. De esta forma, no fue necesario depender de programas ejecutables instalados en el servidor para generar los gráficos.

Referencias bibliográficas

- Joyanes Aguilar, L. (2008). *Estructuras de datos en Java*. Madrid: McGraw-Hill.
- Microsoft. (2026). *Documentación de ASP.NET Core MVC*. Obtenido de Microsoft Learn.
- Takeyas, B. L. (2012). *Estructura de Datos Orientados a Objetos: Pseudocódigo y Aplicaciones en C#.NET*. México: Alfa Omega.
- Weiss, M. A. (2013). *Estructuras de datos y análisis de algoritmos en C++*. Pearson.

Anexos

Semana	Actividad
Semana 1	Creación de la solución Web MVC, arquitectura de clases base (Libro, NodoCategoria, NodoLibro) e implementación del Árbol N-Ario.
Semana 2	Implementación del Árbol AVL con rotaciones de auto-balanceo, métodos In-Order y módulo de Carga Masiva XML incremental.
Semana 3	Desarrollo de vistas, controladores del catálogo, reportes en Graphviz con Viz.js, módulo de Ayuda y pruebas finales.

Tabla 1. Cronograma de Actividades
Fuente: Elaboración Propia

1. Diagrama de Clases

